PRINCIPAL · SEMANA 8 · QUANTIZAÇÃO A FUNDO

Quantização a fundo: FP16 → FP8 → INT4 → FP4

O que quebra e o que acelera quando um número perde bits.
E por que 4× menos bytes quase nunca viram 4× mais tokens por segundo.

FP16 FP8 INT4 FP4

BA Beatriz Almeida · 06/11/2026

A PERGUNTA

4× menos bits por peso. 4× mais rápido?

Qwen2.5-0.5B. Quanto cada token precisa ler da memória:

F16	Q4_0 · “4 bits”	TOK/S
988 MB	346 MB	?
16 bits por peso	2,9× menos por token	subiu 2,9×? 4×?

Guardar cada número com menos bits deixa o arquivo menor. A dúvida de hoje: **arquivo menor faz o modelo escrever mais rápido na mesma medida?**

Sinal, expoente e mantissa

O expoente diz a ordem de grandeza (a faixa). A mantissa diz os dígitos (a precisão). Cada formato divide os bits de um jeito.

FORMATO	BITS	MAIOR NÚMERO	DÍGITOS	USO
FP32	32	**$3{,}4 \times 10^{38}$**	7,2	referência
FP16	16	**65.504**	3,3	inferência
BF16	16	**$3{,}4 \times 10^{38}$**	2,4	treino
FP8 E4M3	8	**448**	1,2	pesos e ativações
FP8 E5M2	8	**57.344**	0,9	gradientes
FP4 E2M1	4	**6**	0,6	MXFP4 / NVFP4



dígitos = casas decimais certas perto de 1 = (bits de mantissa + 1) × $\log_{10}2$

EM PALAVRAS SIMPLES Uma parte diz o tamanho do número (expoente), outra diz os dígitos (mantissa). Menos bits: menos dígitos, ou números menores.

Dois números, seis formatos

O mesmo valor guardado em cada formato. Calculado com o laboratório da semana; arredondamento para o vizinho mais perto.

0,1: um peso típico

testa a precisão

FP32	0,100000001	ok
FP16	0,099976	ok
BF16	0,100098	ok
FP8 E4M3	0,1016	perde
FP8 E5M2	0,0938	perde
FP4 E2M1	0	zera

70.000: uma ativação grande

testa a faixa

FP32	70.000	ok
FP16	+∞ (estourou)	quebra
BF16	70.144	ok
FP8 E4M3	448 (satura)	quebra
FP8 E5M2	+∞ (estourou)	quebra
FP4 E2M1	6 (satura)	quebra

Por isso todo formato de 8 ou 4 bits vem com uma escala: o tensor é dividido por um número antes de arredondar, para caber na faixa e usar bem os degraus.

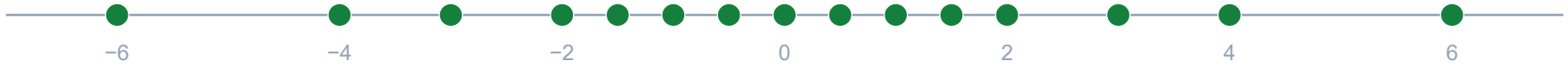
EM PALAVRAS SIMPLES Ou ele perde dígitos (0,1 vira 0,094), ou não cabe (70.000 vira infinito).

FP4 e INT4: 16 códigos, degraus diferentes

Os dois têm 4 bits. O INT4 espalha os 16 valores por igual. O FP4 junta valores perto do zero, onde estão quase todos os pesos.

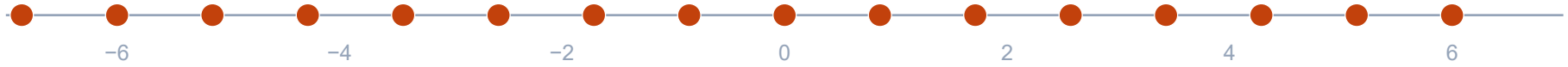
FP4 E2M1

15 valores: ±0; ±0,5 … ±6



INT4

16 inteiros: −8 a 7, × escala 6/7



Perto do zero o FP4 tem degraus de 0,5; o INT4, de 0,86. Pesos seguem uma curva em sino: a maioria é pequena.

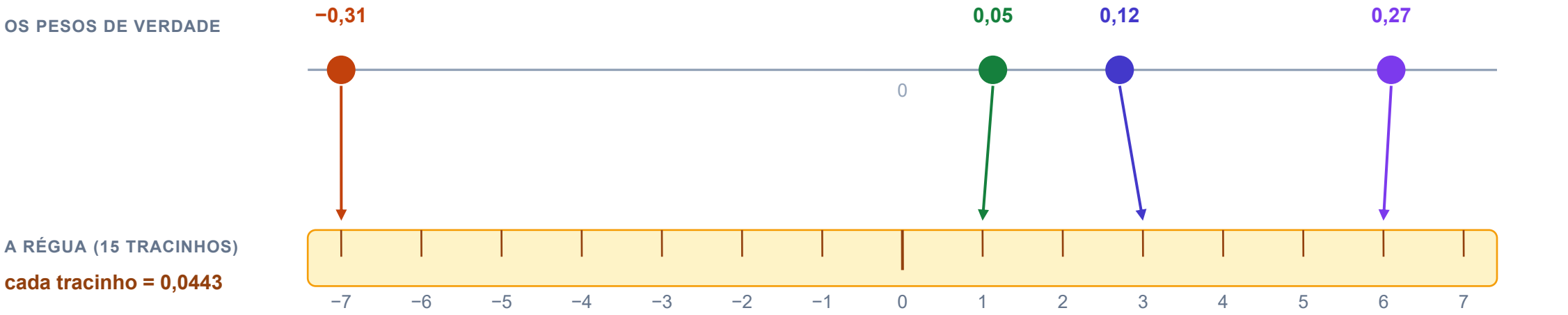
Perto do topo o FP4 salta de 4 para 6; o INT4 continua fino. Com uma escala por grupo, os dois ficam parecidos: no Qwen, 10,3% (INT4) contra 11,7% (MXFP4) de erro.

EM PALAVRAS SIMPLES

Os dois têm só 16 posições para guardar um número. O INT4 espalha as posições como uma régua comum. O FP4 põe mais posições perto do zero, onde ficam quase todos os pesos.

A ideia: uma régua com 15 tracinhos

Antes das contas: escalar e arredondar é pôr cada peso no tracinho mais perto de uma régua feita sob medida.



Vai para o arquivo

os números dos tracinhos: **`3  -7  1  6`** (4 bits cada)
\+ o tamanho do tracinho:
`0,0443` (a escala)

Para usar, volta pela régua

tracinho × tamanho: **`3 × 0,0443 = 0,133`**
era 0,12. Perto, mas não igual: esse é o erro.

EM PALAVRAS SIMPLES

A escala é o tamanho do tracinho da régua. Cada peso vira o número do tracinho mais perto (de −7 a 7), e esse número cabe em 4 bits. Para ler de volta: número × tamanho do tracinho.

2 · A ESCALA

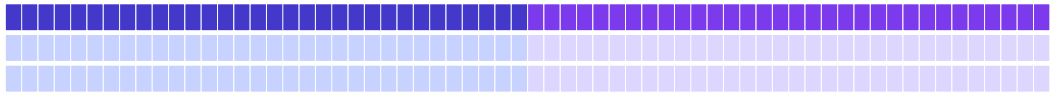
Como 16 degraus guardam qualquer peso

Exemplo com um grupo de 4 pesos guardado em INT4 (inteiros de −7 a 7). Vão para o arquivo só os inteiros e a escala.

1 Pesos originais	2 ÷ escala	3 Arredonda	4 × escala (de volta)
0,12	2,71	3	0,133
−0,31	−7,00	−7	−0,310
0,05	1,13	1	0,044
0,27	6,10	6	0,266
FP16: 16 bits cada	escala = 0,31 ÷ 7 = 0,0443	vai para o arquivo: 4 bits cada	erro de 3,4%

O QUE É UM GRUPO

A matriz de pesos de uma camada é cortada em pedaços de 32 pesos vizinhos da mesma linha. Cada pedaço é um grupo e ganha a sua escala (a sua régua).



grupo 1: 32 pesos → 1 escala grupo 2: 32 pesos → 1 escala

QUANTO A ESCALA CUSTA

32 pesos × 4 bits **= 128 bits**
+ 1 escala de 16 bits **= 144 bits**
144 ÷ 32 pesos **= 4,5 bits por peso**

Por isso o “4 bits” do Q4_0 é, na conta, 4,5.

Um peso gigante estica a régua de todo mundo

Os mesmos pesos pequenos, mais um peso de 8,0. A régua agora precisa ir até 8, e os tracinhos ficam enormes.

RÉGUA DOS PEQUENOS

vai até 0,31



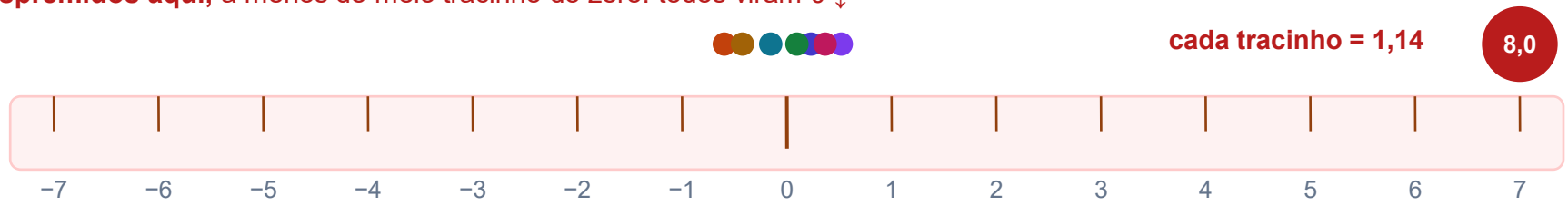
cada peso pequeno cai num tracinho diferente: todos sobrevivem

os 7 pequenos ficam espremidos aqui, a menos de meio tracinho do zero: todos viram 0 ↓

RÉGUA COM O GIGANTE

vai até 8,0

cada tracinho = 1,14



A saída: cada grupo de pesos ganha a sua régua. O gigante só estica a régua do grupo dele; os outros grupos continuam com tracinhos finos.

EM PALAVRAS SIMPLES

Um peso gigante obriga a régua a ir até ele. Os tracinhos ficam tão espaçados que os pesos pequenos caem todos no tracinho do zero. Com uma régua por grupo, o gigante só estraga o grupo dele.

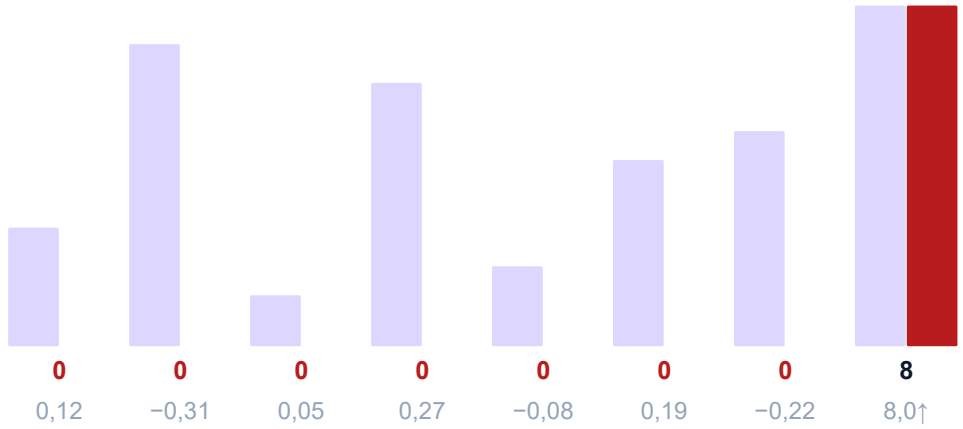
2 · A ESCALA

Um outlier apaga os pesos pequenos

Os mesmos pesos, agora com um oitavo de valor 8,0. Todo LLM tem alguns. Uma escala para o tensor todo contra uma por grupo de 4.

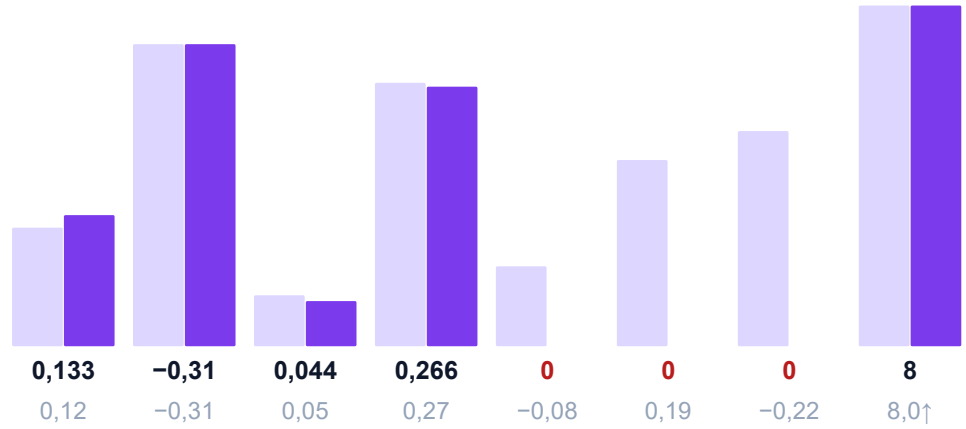
Uma escala para o tensor: 8 ÷ 7 = 1,14

7 dos 7 pesos pequenos viram 0



Uma escala por grupo de 4

o 1º grupo volta inteiro; só o grupo do outlier perde



claro: original · escuro: depois de ida e volta · em vermelho, o que virou zero · barra do 8,0 cortada

No Qwen2.5-0.5B real: o maior |peso| das camadas é 1,688; o peso típico, 0,020. Um é 84× o outro.

EM PALAVRAS SIMPLES

Se um peso é gigante, a régua fica enorme para ele caber, e os pesos pequenos caem todos no primeiro tracinho: viram zero. Uma régua para cada grupinho de pesos resolve.

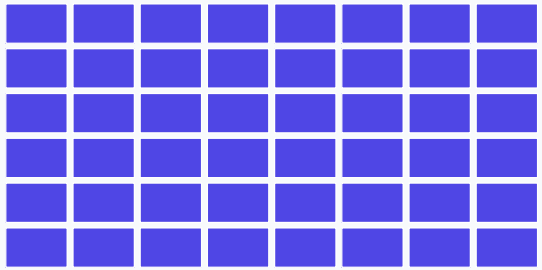
2 · A ESCALA

Quantas escalas: tensor, canal ou grupo

Mais escalas protegem contra outliers e custam bits. Medido nos 358 M pesos das camadas do Qwen2.5-0.5B, em INT4.

Por tensor

1 escala para a matriz inteira



BITS POR PESO

4,00

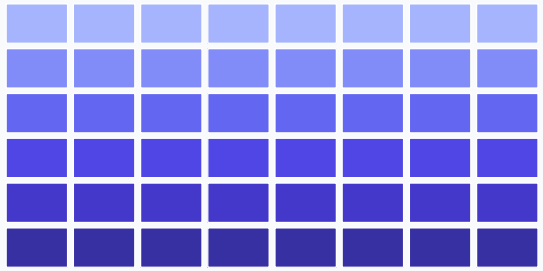
ERRO NOS PESOS

75,8%

quebra

Por canal

1 escala por linha (canal de saída)



BITS POR PESO

4,01

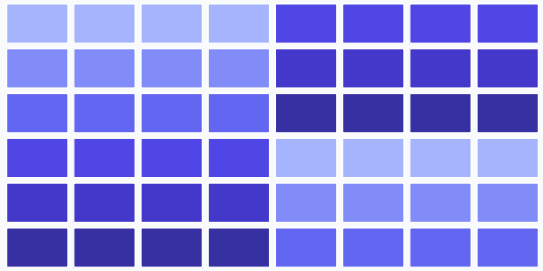
ERRO NOS PESOS

17,9%

perde muito

Por grupo de 32

1 escala a cada 32 pesos da linha



BITS POR PESO

4,50

ERRO NOS PESOS

10,3%

usável

Cada cor é uma escala diferente. Erro = |pesos − pesos de volta| ÷ |pesos|, somado nas 168 matrizes das camadas.

EM PALAVRAS SIMPLES Uma régua para a matriz inteira, uma por linha, ou uma a cada 32 números. Mais réguas, menos erro. Mas cada régua também ocupa espaço no arquivo.

Como cada receita empacota 32 pesos

Cada caixinha é um peso; a largura é quantos bits ele ocupa. A caixa escura é a régua (escala) do grupo, que também ocupa espaço.

Tamanho do grupo = quantos PESOS dividem uma régua: 16 ou 32

Tamanho da régua = quantos BITS ela ocupa: 8 ou 16. Cada peso segue com 4.

BITS POR PESO

FP8 E4M3
pesos de 8 bits



32 × 8 bits = 256 bits (a régua é uma só para a matriz inteira)

8,0

Q4_0
inteiros de 4 bits



32 × 4 bits = 128 + régua de 16 bits = 144 bits → 144 ÷ 32

4,5

MXFP4
FP4 de 4 bits



32 × 4 bits = 128 + régua de 8 bits = 136 bits → 136 ÷ 32

4,25

NVFP4
FP4 de 4 bits



2 grupos de 16 pesos, cada um com régua de 8 bits = 144 bits → 144 ÷ 32

4,5

EM PALAVRAS SIMPLES

Grupo menor erra menos, mas paga mais réguas: por isso “4 bits” vira 4,25 ou 4,5.

Quatro receitas usadas de verdade

Todas guardam cada peso com poucos bits e põem uma régua em cada grupo de pesos. Muda o tamanho do grupo e o tipo de régua.

FP8 E4M3

8 bits

O NOME QUER DIZER

FP8 = ponto flutuante de 8 bits. E4M3 = 4 bits de expoente e 3 de mantissa, mais o sinal.

COMO GUARDA

cada peso: 8 bits
1 régua para a matriz toda

RODA RÁPIDO EM

GPUs NVIDIA H100 e RTX 40

8,0 bits por peso

Q4_0 · Q4_K

llama.cpp

O NOME QUER DIZER

Q4 = quantizado em 4 bits. _0 = o tipo mais simples. Q4_K = versão com grupos dentro de grupos maiores.

COMO GUARDA

cada peso: inteiro de 4 bits
1 régua a cada 32 pesos

RODA RÁPIDO EM

CPU, Mac, qualquer GPU (Ollama, LM Studio)

4,5 bits por peso

MXFP4

padrão aberto

O NOME QUER DIZER

MX = microscaling: uma escala para cada pedacinho. FP4 = ponto flutuante de 4 bits.

COMO GUARDA

cada peso: FP4 de 4 bits
1 régua a cada 32 pesos

RODA RÁPIDO EM

GPUs NVIDIA Blackwell; pesos do gpt-oss

4,25 bits por peso

NVFP4

NVIDIA

O NOME QUER DIZER

NV = NVIDIA. FP4 = ponto flutuante de 4 bits. A régua é mais precisa (FP8) e o grupo, menor.

COMO GUARDA

cada peso: FP4 de 4 bits
1 régua a cada 16 pesos

RODA RÁPIDO EM

GPUs NVIDIA Blackwell (B200, RTX 50)

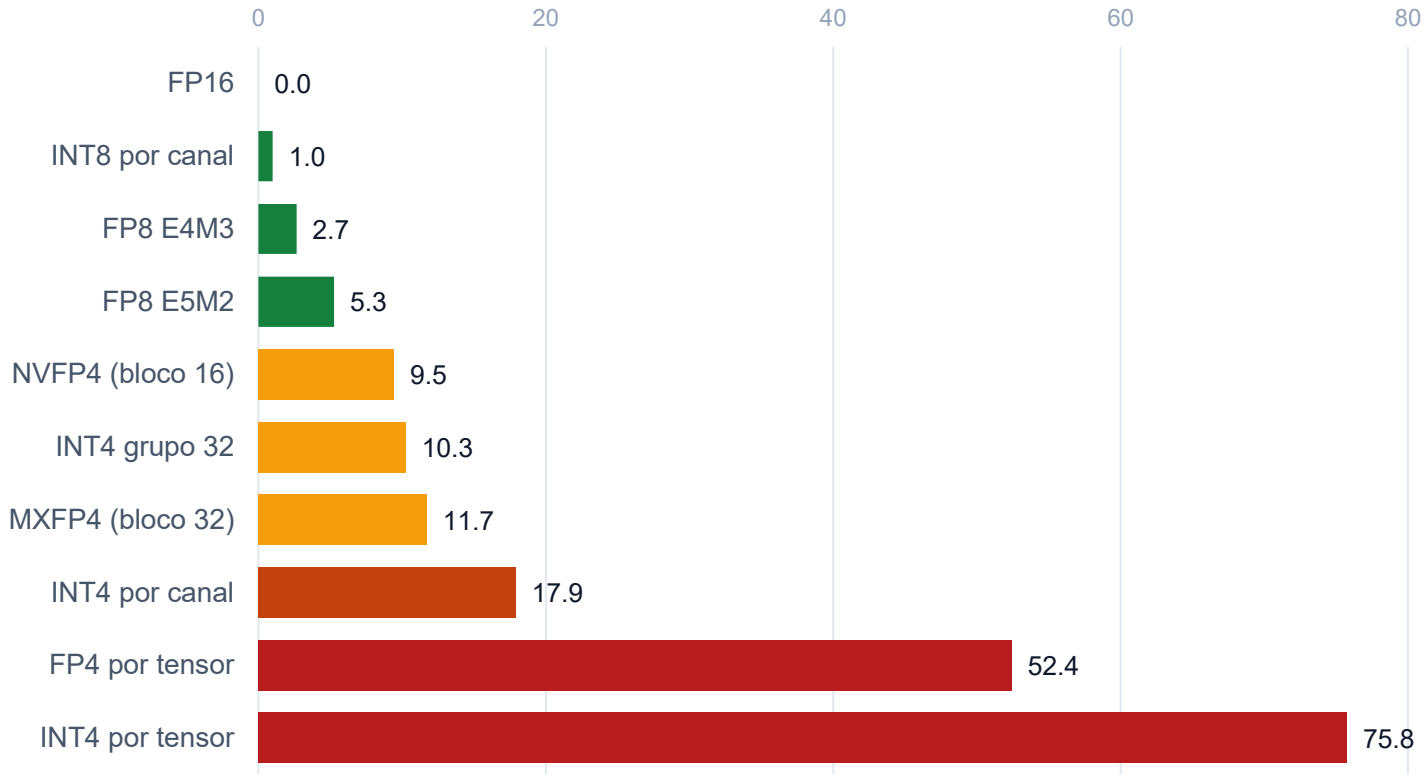
4,5 bits por peso

EM PALAVRAS SIMPLES

As quatro fazem a mesma coisa: números pequenos mais uma régua por grupo. Mudam o tamanho do grupo e o tipo de régua, e cada uma roda rápido num hardware diferente.

Quanto cada formato muda os pesos

Os 358 M pesos das camadas do Qwen2.5-0.5B, arredondados em cada formato. Erro relativo, em %.



COMO LER

Cada bit a mais vale ~6 dB de sinal limpo: o erro cai pela metade.

8 bits, qualquer um, ficam abaixo de 6%. FP8 E4M3 perde mais que INT8 por canal porque usa uma escala só.

4 bits só funcionam com grupos. Sem grupo, mais da metade do sinal some.

EM PALAVRAS SIMPLES

Cada barra mostra o quanto os números do modelo mudaram depois de encolhidos. Barra curta: quase igual ao original. Com 8 bits quase nada muda; com 4 bits, só dá certo usando grupos.

Quando quebra, não é aos poucos

Mesmo modelo, mesmos 4 bits por peso, mesma pergunta. Só muda quantas escalas existem.

INT4 GRUPO 32 · 4,5 bits · perplexidade 22,7

```
> Quanto é 12 vezes 12? Resposta:
 144. Como sabemos que 12

> def fatorial(n):
    if n == 0:
        return 1
```

INT4 POR TENSOR · 4,0 bits · perplexidade 37 milhões

```
> Quanto é 12 vezes 12? Resposta:
 □aller volatileJNIEXPORT'])? shading

> def fatorial(n):
 tàvisoToListAsync□名%n"""),
%ültült主义{name鸣
```

0,5 bit de diferença por peso (as escalas dos grupos)

Não é ruído: cada camada erra um pouco e os erros se somam nas 24 camadas

Teste antes de medir: tok/s de um modelo quebrado não vale nada

EM PALAVRAS SIMPLES Mesma pergunta, mesmos 4 bits. A única diferença é ter uma régua por grupo. Com ela, o modelo responde 144. Sem ela, solta letras sem sentido.

O que mais quebra, além dos pesos

Até aqui só os pesos perderam bits. Num servidor, outras três coisas também podem ser quantizadas, e cada uma tem o seu ponto fraco.

1 Ativações

Têm outliers piores que os pesos, e mudam a cada token.

O que se faz: LLM.int8() separa os canais grandes; SmoothQuant passa a dificuldade das ativações para os pesos.

2 KV cache

Cresce com a conversa. Erros nele afetam todos os tokens seguintes, e mais em contextos longos.

O que se faz: FP8 ou INT8 no KV é comum (vLLM, llama.cpp); 4 bits pede cuidado.

3 Modelos pequenos

Têm menos pesos repetindo a mesma informação.

O que se faz: Para SLMs, 8 bits ou 4 bits com método melhor (GPTQ, AWQ).

4 Tarefas exatas

Conta, código e fatos são as primeiras a errar. A perplexidade média esconde isso.

O que se faz: Avaliar na tarefa de verdade, não só perplexidade (semana 13).

EM PALAVRAS SIMPLES Não são só os pesos que podem ser encolhidos: os valores que passam pelo modelo e a memória da conversa também. E modelos pequenos sofrem mais com isso.

Arredondar sozinho ou compensando

Três pesos de 0,4 e uma régua que só tem 0 e 1. A camada soma os três: o resultado certo é 1,2.

Cada um por si (RTN)

arredonda cada peso sem olhar os outros

PESO	ARREDONDA	SOBRA
0,4	0	perde 0,4
0,4	0	perde 0,4
0,4	0	perde 0,4

soma: **0** (certo: 1,2) erro: **1,2**

Compensando (a ideia do GPTQ)

a sobra de um peso vai para o próximo

PESO	ARREDONDA	SOBRA
0,4	0	passa +0,4
0,4 + 0,4 = 0,8	1	passa −0,2
0,4 − 0,2 = 0,2	0	sobra 0,2

soma: **1** (certo: 1,2) erro: **0,2**

EM PALAVRAS SIMPLES

Arredondar cada peso sozinho deixa os erros se somarem. Os métodos espertos passam o erro de um peso para o próximo, e no fim os erros quase se cancelam. O arquivo continua do mesmo tamanho.

Arredondar melhor: GPTQ, AWQ e QAT

Tudo que medi até aqui é arredondamento simples (round-to-nearest). Os métodos usados de verdade erram menos com os mesmos bits.

RTN

arredondar para o mais perto

O que este deck mediu. Rápido, sem dados. Bom em 8 bits, fraco em 4.

PRECISA DE

nenhum dado

GPTQ

Frantar et al., 2022

Arredonda um peso de cada vez e corrige os seguintes para compensar o erro, usando algumas frases de calibração.

PRECISA DE

~128 frases

AWQ

Lin et al., 2023

Acha os ~1% de pesos que mais importam (pelas ativações) e aumenta a escala deles antes de arredondar.

PRECISA DE

~128 frases

QAT

treino com quantização

Treina (ou ajusta) o modelo já arredondando no caminho, para ele aprender a conviver com o erro.

PRECISA DE

treino

Nenhum muda a velocidade: o arquivo final tem o mesmo formato. Mudam quais inteiros e escalas vão para ele. QAT e QLoRA são a semana 9 (Pedro).

EM PALAVRAS SIMPLES

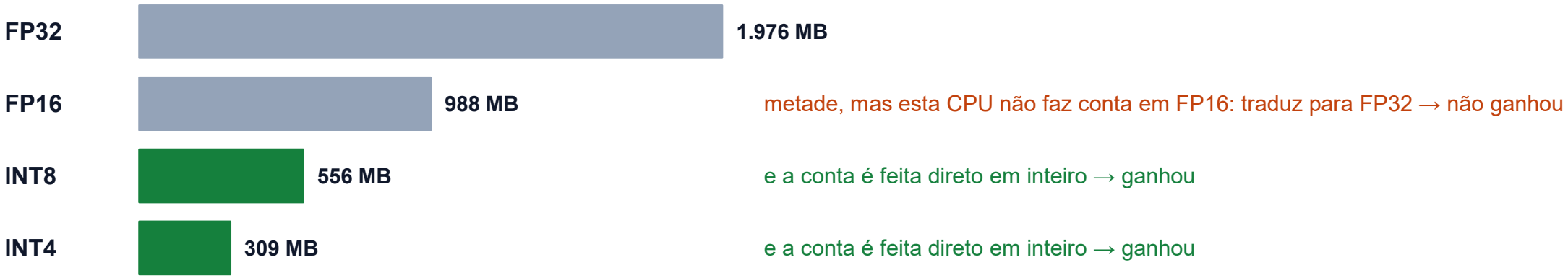
Em vez de só arredondar cada número, esses métodos arredondam de um jeito esperto, para o erro de um compensar o do outro. A qualidade melhora; a velocidade fica igual.

Para cada palavra, o modelo inteiro passa pelo cano

No decode, cada token novo lê todos os pesos da memória. O cano entre a memória e o processador tem largura fixa.



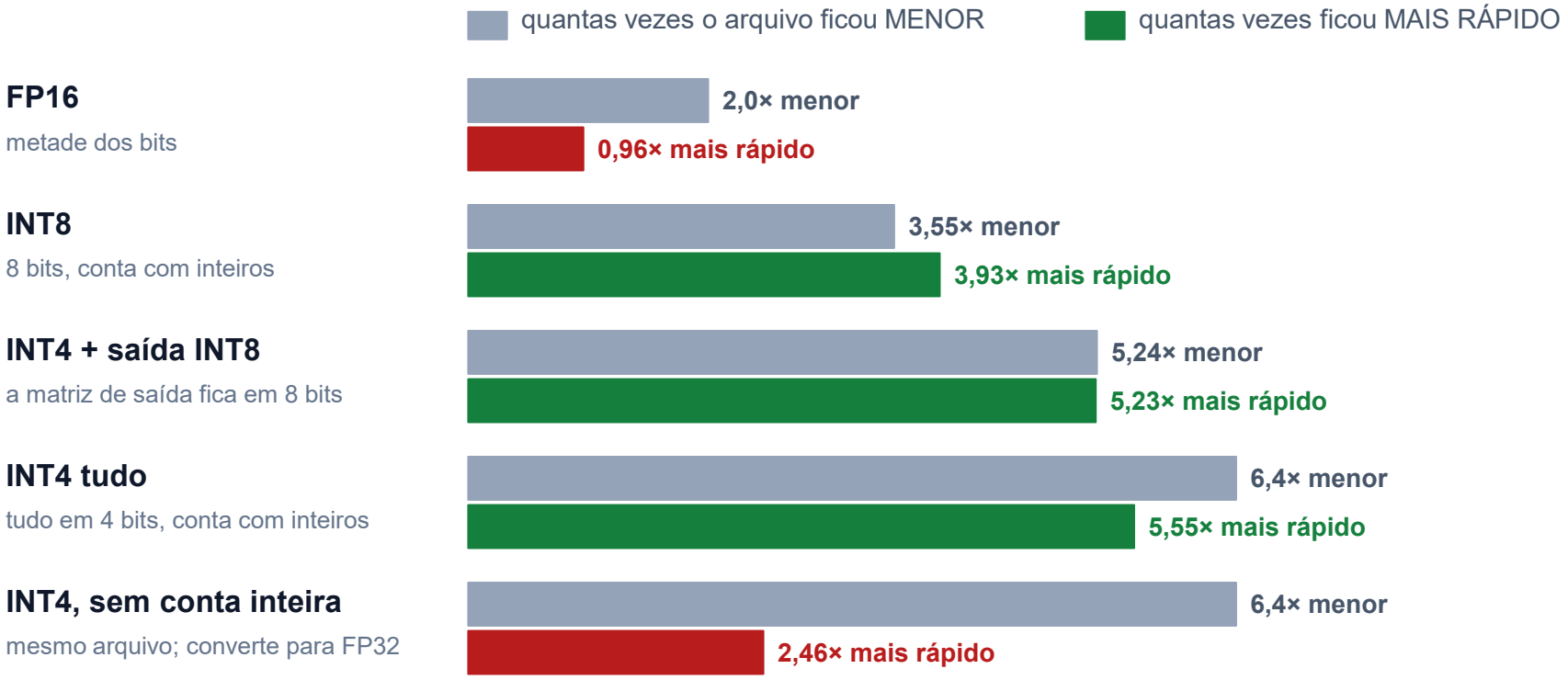
QUANTO PASSA PELO CANO A CADA TOKEN



EM PALAVRAS SIMPLES Escrever uma palavra = passar todos os pesos pela memória. Pesos menores passam mais rápido, desde que o processador saiba usar o formato que chegou do jeito que chegou.

Arquivo menor ficou mais rápido? Medido no meu PC

Só as contas de um token com os pesos (Qwen2.5-0.5B, ONNX Runtime, CPU de notebook), comparadas com o FP32 original.



Referência: o FP32 faz 14,1 tok/s (palavras por segundo, só contando estas contas). INT4 tudo faz 78,1.

COMO LER

Barras do mesmo tamanho: a economia de memória virou velocidade (INT8, INT4).

Barra de baixo bem menor: algo comeu o ganho.

FP16: esta CPU não sabe fazer conta em FP16 e converte tudo para FP32.

INT4 sem conta inteira: converte cada peso para FP32 antes de multiplicar.

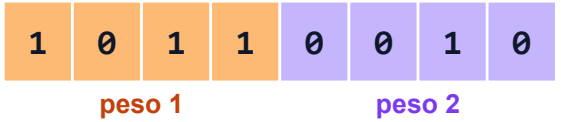
EM PALAVRAS SIMPLES

Barras do mesmo tamanho: a economia de memória virou velocidade. Barra de baixo menor: o processador perdeu tempo convertendo números e comeu o ganho.

Por que o mesmo INT4 tem três velocidades

Dois pesos de 4 bits viajam colados num byte. Antes da conta, alguém precisa desembalar. O que muda é quanto trabalho vem depois.

1 BYTE = 2 PESOS COLADOS



Desembalar = separar as duas metades e dar a cada uma o seu número de tracinho. Todo kernel faz isso; o que vem depois é o que diferencia.

PyTorch int4pack	desembala	→ caminho lento para 1 token nesta CPU	→ multiplica	**0,05×**
ONNX Runtime, nível 0	desembala	→ converte cada peso para FP32	→ multiplica em FP32	**2,5×**
ONNX Runtime, nível 4	desembala	→ multiplica em inteiro (VNNI)		**5,6×**

tok/s contra FP32 (medido)

EM PALAVRAS SIMPLES Dois pesos de 4 bits viajam colados num byte e precisam ser desembalados. Depois disso, quanto menos trabalho até a conta, mais rápido. O arquivo é o mesmo nos três casos.

O mesmo INT4, três velocidades

Os mesmos 4 bits por peso, a mesma CPU. O que muda é o kernel: o código que desempacota e multiplica.

0,05×

tok/s contra FP32

PyTorch 2.7, int4pack na CPU

20× mais LENTO que FP32

O kernel não é otimizado para lote 1 nesta CPU. Ler 6× menos bytes não ajuda quando o processador passa o tempo desempacotando.

2,5×

tok/s contra FP32

ONNX Runtime, nível 0

desempacota para FP32

Cada peso de 4 bits vira FP32 antes de multiplicar. A memória fica livre, mas o processador vira o gargalo.

5,6×

tok/s contra FP32

ONNX Runtime, nível 4

conta em inteiro (INT8)

A ativação vira INT8 e a multiplicação é inteira, com instruções VNNI. O trabalho extra quase some.

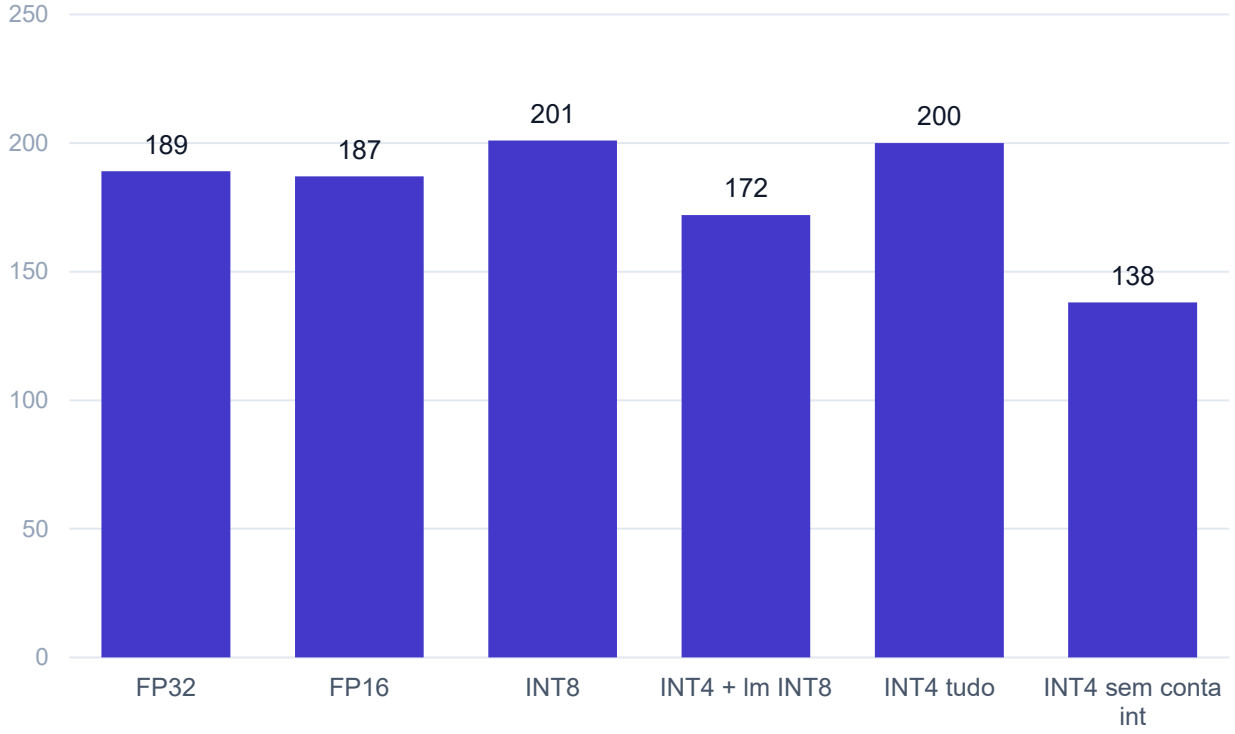
Formato é arquivo; velocidade é kernel. Por isso a mesma GGUF roda em velocidades diferentes em cada runtime e cada máquina.

EM PALAVRAS SIMPLES

Os números de 4 bits vêm embalados e precisam ser desembalados antes da conta. O mesmo arquivo fica lento ou rápido dependendo de quem escreveu o programa que desembala.

No prefill, guardar em 4 bits não acelera

Mesmo grafo, agora com 64 tokens por passo (como ao ler um prompt). Cada peso trazido da memória é usado 64 vezes.



Tokens processados por segundo, lote de 64

POR QUÊ

Com 64 tokens, o processador faz 64× mais contas por byte lido. O limite passa a ser **multiplicar**, e guardar em menos bits não diminui as contas. Desempacotar é até trabalho a mais.

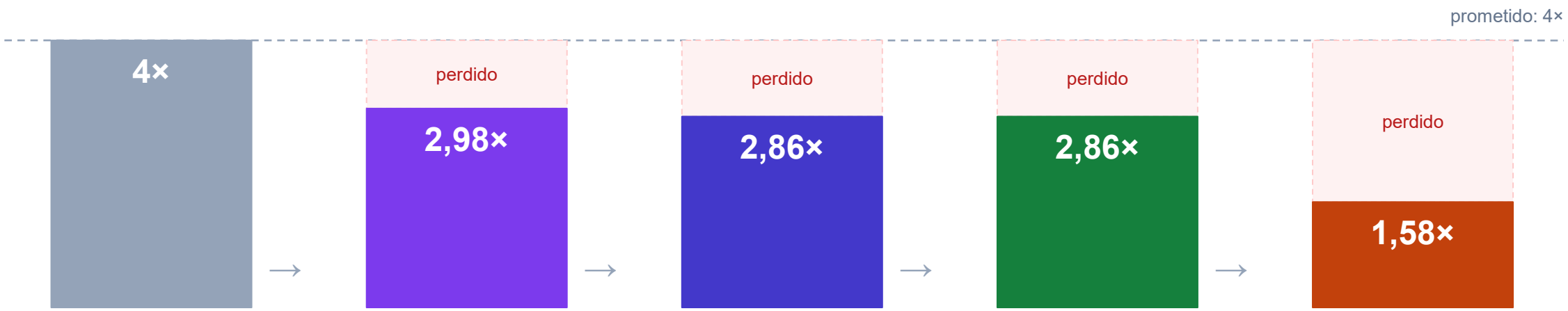
O QUE ACELERA O PREFILL

Calcular em menos bits, não só guardar. Na H100, FP8 faz **~2.000 TFLOPS** contra ~1.000 em BF16. A Blackwell dobra de novo com FP4. CPU comum não tem nada disso.

EM PALAVRAS SIMPLES Ao ler a pergunta, o modelo processa muitas palavras de uma vez. Aí o trabalho pesado é fazer contas, não ler números da memória, e números menores não diminuem as contas.

Do nome ao token: onde o ganho some

De F16 para Q4_0 no Qwen2.5-0.5B. Prometido: 4× mais rápido. Cada coluna mostra quanto ainda sobra; o vermelho é o que já se perdeu.



1. O nome promete

16 bits → 4 bits: 4× menos. Seria 4× mais rápido.

2. O arquivo de verdade

Cada grupo de 32 pesos leva uma régua (+0,5 bit) e a matriz de saída fica em 8 bits.

3. O que cada token lê

Conta só o que é lido a cada palavra: 988 MB → 346 MB.

4. Só as contas

Com conta em inteiro, o tempo das contas caiu junto com os bytes.

5. A palavra inteira

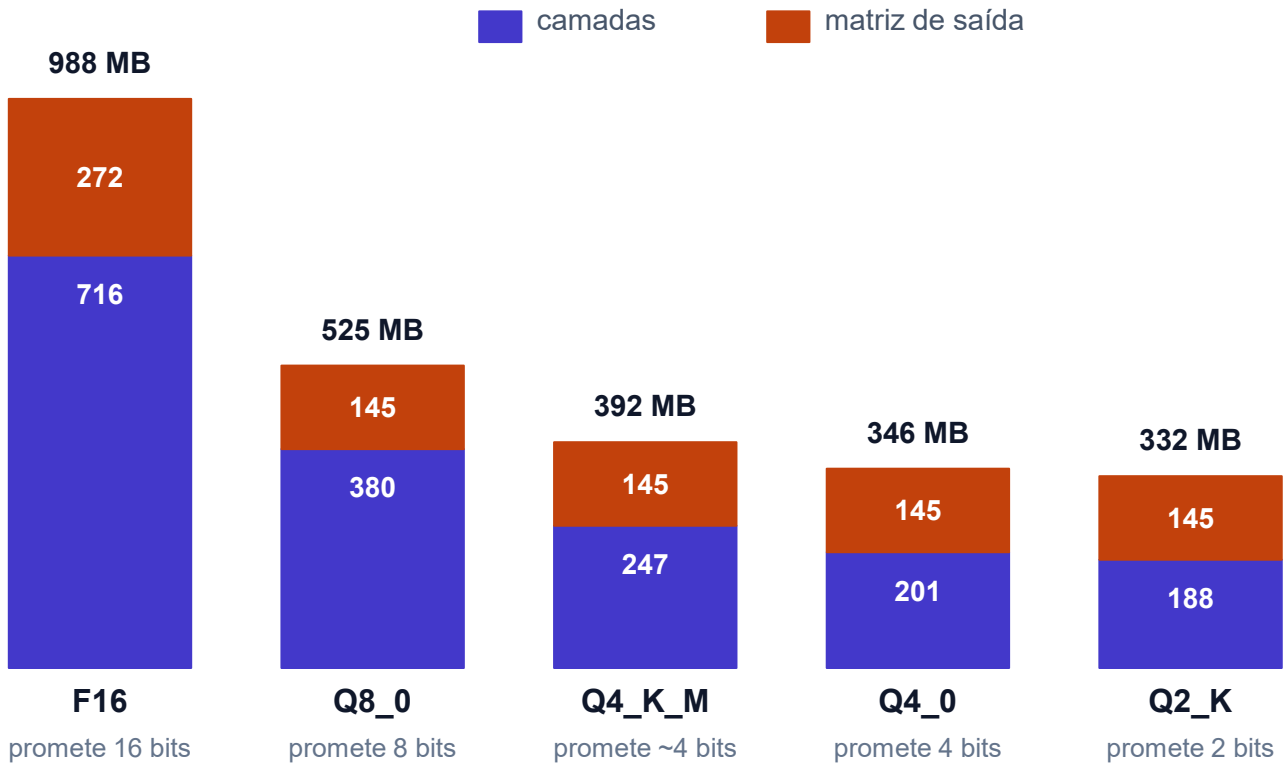
O resto do trabalho (29 ms por palavra) não encolheu.

EM PALAVRAS SIMPLES

Prometido: 4× mais rápido. O vermelho é o ganho que foi sumindo: as réguas, a parte que fica em 8 bits e o trabalho que não encolhe. Sobrou 1,6×.

Dentro dos arquivos GGUF: a parte que não encolhe

GGUF é o arquivo de modelo do llama.cpp e do Ollama. Abri os 5 arquivos oficiais do Qwen2.5-0.5B: quantos MB cada palavra lê?



Azul: as camadas

Os pesos que fazem o modelo “pensar”. Encolhem quando os bits caem: 716 → 201 MB.

Vermelho: a matriz de saída

A tabela que escolhe a próxima palavra entre 151.936. Fica em 8 bits em todos: 145 MB parados.

O “2 bits” quase empata

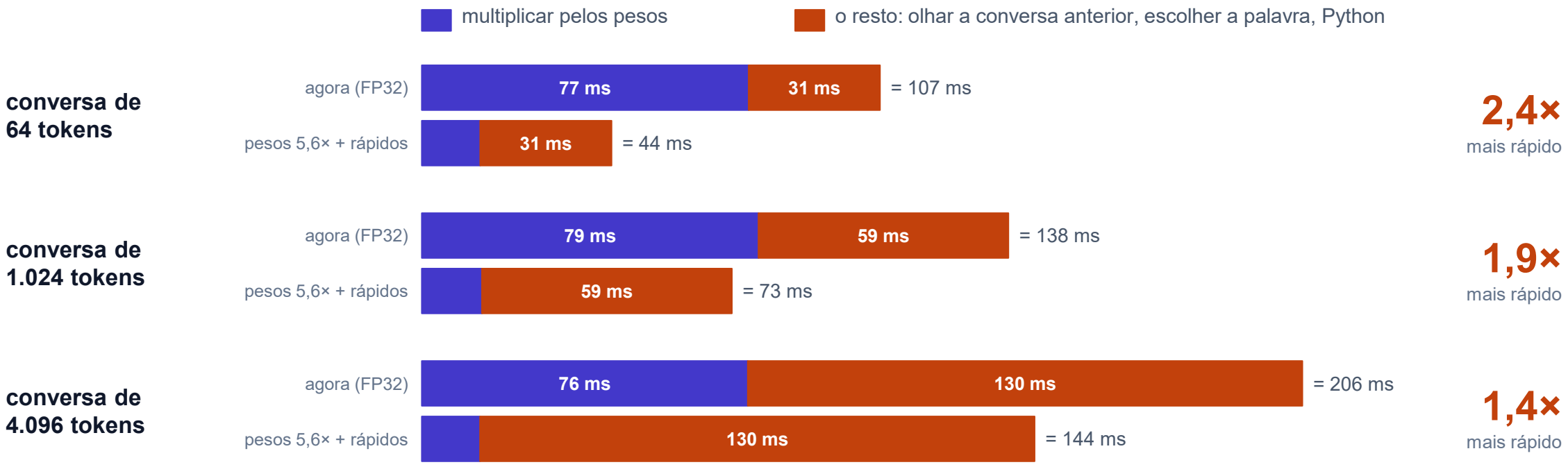
O Q2_K guarda pesos em blocos de 256, e as linhas do Qwen têm 896. Onde não cabe, volta para 3 e 4 bits: 332 MB contra 346.

EM PALAVRAS SIMPLES

Abri os arquivos para ver o que cada um guarda. A parte azul encolhe; a vermelha (a tabela do vocabulário) fica do mesmo tamanho. Por isso o arquivo de 2 bits é quase igual ao de 4.

Num modelo de verdade, 29% a 63% do token não é peso

Cronometrei o Qwen2.5-0.5B escrevendo palavras (FP32). Azul: multiplicar pelos pesos. Laranja: todo o resto.



O azul encolhe 5,6×, o laranja não muda. Quanto mais longa a conversa, maior o laranja e menor o ganho.

EM PALAVRAS SIMPLES

Quantizar acelera só o azul. O laranja (olhar a conversa e escolher a palavra) continua igual, então a palavra inteira acelera bem menos que 5,6×.

A RESPOSTA

O tok/s aumentou na mesma proporção em que os bytes por token caíram?

Não. F16 → Q4_0 três coisas consumiram o ganho:

1 · OS BYTES CAEM MENOS QUE O NOME

Escalas e a saída em 8 bits

Cada grupo de 32 guarda uma escala (+0,5 bit) e a matriz de saída fica em Q8_0: 145 MB, 42% do que o token lê. 4× vira 2,9×.

2 · O KERNEL TEM QUE APROVEITAR

Desempacotar é trabalho

Com conta inteira, as multiplicações acompanharam os bytes (5,6×). Desempacotando para FP32, 2,5×. Com kernel ruim, 20× mais lento. FP16 nesta CPU: nenhum ganho.

3 · O RESTO NÃO ENCOLHE

29% a 63% do token

Atenção sobre o KV cache, normalizações, escolha do token e o runtime não mudam com o formato. E crescem com o contexto: 2,4× vira 1,4× com 4 mil tokens.

E no prefill, nada: com 64 tokens por passo, quem limita são as contas, e guardar em 4 bits não diminui contas. Acelerar ali pede calcular em FP8 ou FP4, o que só GPUs novas fazem.

Medido no notebook-bia (i5-1135G7, só CPU), Qwen2.5-0.5B. O ~1,6× junta duas medidas: o tempo das multiplicações (ONNX Runtime) e o resto do token (transformers).

EM PALAVRAS SIMPLES Encolher o arquivo 4× não deixa o modelo 4× mais rápido: parte do arquivo não encolhe, desembalar os números dá trabalho e o resto do serviço continua do mesmo tamanho.